CORE INVOICE-MANAGEMENT ENDPOINT · FACTORING PLATFORM

~~242 s~~ → 12 s

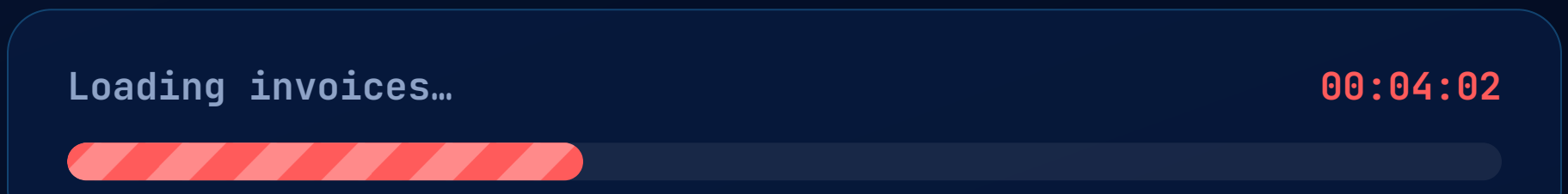
The endpoint **29 operations staff** waited on every day — from four minutes to twelve seconds.



TAKEN ON AS A JUNIOR DEVELOPER

C# · .NET | SQL SERVER · SSMS | REACT

FOUR MINUTES. EVERY LOAD. EVERY DAY.



29
operations staff using it daily

242s
per load, at its worst

#1
the core of the factoring product

It lived in the company's **heaviest monolith**, and the debt had been there for a long time. Product leadership made it a priority.

> In sprint planning I volunteered. I was a junior, but I had the most time on the product in a team of two juniors and one mid.

I WAS LOOKING IN THE ~~WRONG~~ PLACE.

× WHAT I ASSUMED

logic inside controllers

repeated methods

hard-coded values

no clear responsibilities

"IT'S THE BACKEND CODE."

✓ WHAT DEBUGGING SHOWED

the code was a mess, yes

but the weight was in the DB

stored procedures doing the heavy lifting

over the product's largest tables

"IT'S THE DATA PATH."

My first pass went straight into the code — **no logs, no look at the tables, no check of the frontend.** A full debugging session corrected the diagnosis.

Then I wrote a **step-by-step restructuring plan** and validated it with the interim tech lead before changing a line.

FOUR STEPS, IN THIS ORDER.

01

CLEAR THE PATH TO THE DATABASE

Removed inefficient call loops, added validations to stop endless loads, and — with the acting product owner — cut deprecated business logic. About 40% of the backend.

02

FIX THE DATA LAYER

Rewrote the heaviest stored procedures, and wrote new ones where other modules still depended on the old. Added missing indexes; dropped unused ones only after checking every dependency.

`≈ -65% latency on procedure calls`

03

FINISH THE BACKEND

The remaining 60% of the restructure, each endpoint tested through Swagger.

04

REDESIGN THE FRONTEND

With the UX/UI lead: pagination, new filters, clearer feedback. Business validations moved to the backend. Signed off by the CPO.

MEASURED, NOT FELT

−95%

LOAD TIME

242s → 12s

QUERY COST

~125K → ~2,300ms

LOGICAL READS

~4.5M → ~23K

MEMORY

~2.1 GB → ~420 MB

CONCURRENT USERS, NO DEGRADATION

5 → 30

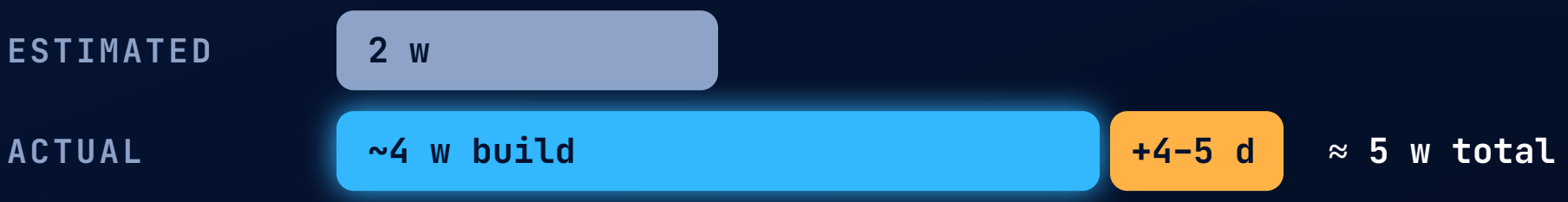
HOW IT WAS MEASURED

DB SQL Server statistics and Query Store, before and after — run with the interim tech lead.

UX End-user stopwatch: Staging with the fix vs Production without it, side by side.

PROD Final check in production: 242 s → 12 s.

ESTIMATED: 2 WEEKS. REAL: ~5.



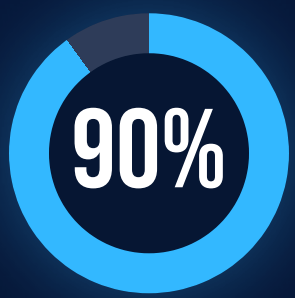
THE 2 WEEKS WERE A GUESS, MADE BEFORE ANY ANALYSIS · 4 WEEKS OF BUILD + 4–5 DAYS WAITING FOR PRODUCT SIGN-OFF

WHAT I'D DO DIFFERENTLY

I left the frontend for last. Moving its business validations to the backend meant **reopening a backend I had just finished** — more code, more tests.

Today I map the **whole path, UI included,** before changing the first line.

MY PART



Analysis, plan, build and fix. The interim tech lead validated the plan and ran the DB measurements with me.

THE OUTCOME

The fix reached **the CEO**. Together with my habit of asking for the hardest tickets, it led to my **promotion to Mid-Level** soon after.

RODRIGO SERRATO GOMEZ

FULL STACK ENGINEER · TYPESCRIPT · NESTJS · .NET · SQL SERVER

linkedin.com/in/rodrigo-rafael-serrato-gomez

Open to US B2B contracts